

FPL H2H League Dashboard — Product Requirements Doc

1. Overview

A live, mobile-friendly web dashboard for a private 10-person FPL Head-to-Head league. Pulls data from the official FPL API, displays standings, live gameweek matchups, and historical results — styled loosely after official Premier League / FPL app dashboards (clean cards, team badges, bold scorelines).

Not a native app. A responsive website, hosted on Vercel, viewed in mobile Safari/Chrome like any site. Bookmarked, not installed (though it could later be a PWA if you want an app-icon feel — flagging as optional, not in scope for v1).

2. Tech Stack

Layer	Choice	Why
Framework	Next.js (App Router) + React + TypeScript	Frontend + backend API routes in one project, deploys natively to Vercel
Styling	Tailwind CSS	Fast, responsive-by-default, easy to theme (PL-style dark/purple/green accents)
Database	Supabase (hosted Postgres)	Free tier, built-in realtime subscriptions, simple client SDK
Hosting	Vercel	Free tier, auto-deploys from GitHub, cron jobs for polling
Data source	Official FPL API (<code>fantasy.premierleague.com/api/</code>)	Free, no auth, all league/player/live data

Data flow: Vercel Cron hits FPL API on a schedule → writes to Supabase → frontend reads from Supabase (via Supabase client, optionally realtime-subscribed) → nobody's browser

hits FPL directly, so no rate-limit risk and instant page loads.

3. Core Data Model (Supabase tables)

- `managers` — `id`, `fpl_entry_id`, `display_name`, `team_name`, `draft_order` (nullable)
 - `gameweeks` — `id`, `deadline_time`, `is_current`, `is_finished`
 - `h2h_matches` — `id`, `gameweek_id`, `manager_1_id`, `manager_2_id`, `score_1`, `score_2`, `winner_id` (nullable until finished)
 - `standings_snapshot` — `manager_id`, `gameweek_id`, `points`, `wins`, `draws`, `losses`, `league_rank` (a row per manager per GW, so you get a historical rank-over-time chart)
 - `live_points_cache` — `manager_id`, `gameweek_id`, `current_points`, `last_updated` (the fast-changing table, overwritten each poll)
-

4. Pages / Tabs

1. Standings (home tab)

- Full-season H2H league table, **styled like a real Premier League table**: rank number, manager, played, W-D-L, points column bolded on the right, promotion/relegation-style color bar if you want zones (e.g., top 3 highlighted)
- League points (3 for a win, 1 for a draw, 0 for a loss) — since the real FPL league stays Classic, this table is calculated by the dashboard itself: pull each manager's GW score from the Classic league / `entry/{id}/event/{gw}/picks`, compare scores against our own generated schedule (section 7), and tally W-D-L + points ourselves. This mirrors what FPL's native `leagues-h2h/standings` would show, just computed on our end since we're not using their H2H league type.
- **Auto-updates**: table recalculates automatically as each gameweek's scores finalize — no manual entry, ever
- Row highlight for live gameweek in progress
- Tap a manager → mini profile (team name, current squad thumbnail, season trend)

2. This Gameweek

- Matchup cards, one per pairing: "Manager A [live score] vs [live score] Manager B"

- Green/red glow on the currently-winning side, live-updating
- Expand a card → both squads side-by-side, player-by-player points

3. Fixtures / Schedule

- Full-season H2H matchup schedule (who plays who, which GW) — generated once, shown read-only

4. Draft Order / Randomizer tab — REMOVED. No redraft is happening; this tab isn't needed. The custom H2H schedule (section 7) is generated once behind the scenes at setup, not a user-facing randomizer feature.

4. Manager Profile

- Season history: rank over time (line chart), best/worst GW, head-to-head record vs each other manager

5. Live Update Behavior

- During an active gameweek: Vercel Cron polls FPL's live endpoint every 60–120 seconds, writes to `live_points_cache`
- Supabase realtime pushes the change to any open dashboard — no manual refresh needed
- Outside gameweeks: no polling, dashboard just reads last-finalized data
- Bonus points can shift for ~1 hour after matches end — UI should show a small "provisional" tag on GW scores until FPL marks the gameweek `is_finished`

6. Visual Style

- Dark background, single accent color per manager (auto-assigned, consistent across pages) for quick visual scanning
 - Card-based layout (not dense tables) on mobile; table view can appear on wider/desktop screens
 - Team crests already exist in FPL's bootstrap data (`teams` array has badge codes) — usable for real club badges in each squad view
-

7. H2H Scheduling Algorithm (matches real FPL H2H behavior)

Since FPL doesn't publish its exact scheduling source code, this replicates its documented behavior as closely as possible:

- Real FPL H2H generates a schedule **once, at league lock**, using a single round-robin — it does not re-randomize week to week
- It runs as a **double round-robin** across the season, with the second half mirroring the first (reverse fixtures roughly from the season's midpoint onward)
- **Our version:** generate one round-robin for GW1–9 using the standard **circle method** (fixes one manager, rotates the rest — the same algorithm real round-robin schedulers use), with the initial pairing order **randomized once** at setup to mimic FPL's random initial draw
- That same 9-round pattern is then **repeated** for the remainder of the 38-week season (GW10–18, 19–27, 28–36, GW37–38) — no wildcard/final logic, since real FPL H2H doesn't do special playoff gameweeks either; it just runs the standard format straight through to GW38
- Result: everyone faces everyone in a fixed, fair rotation that mirrors what native FPL H2H would have produced, just running on your own Classic league's scores underneath

8. PL Calendar / Fixture Sync

- No separate calendar integration needed — FPL's own API carries real Premier League fixture data:
 - `bootstrap-static → events[].deadline_time` (GW deadlines)
 - `fixtures/?event={gw}` → real match kickoff times, home/away teams, per gameweek
- Dashboard surfaces:
 - Live GW deadline countdown
 - Mini fixture ticker per gameweek (which real PL matches are on, so managers know when their players are active)
 - "Add to Calendar" button generating a downloadable `.ics` per GW deadline (phone calendar reminders, not a live sync — deadline data doesn't change once fixtures are set, so no ongoing sync needed)

9. Visual Polish (“make it look cool”) + Performance Priority

- **Speed and glanceability over features** — this dashboard exists specifically to be faster and lighter than the official FPL app; every design decision favors quick-glance clarity over visual complexity
- Live “pulse” indicator (small animated dot) on cards with a match currently in play
- **Auto-assigned per-manager accent color** (no manual color picker — one less setup step), consistent across all pages, for fast visual scanning
- Bold PL-broadcast-style scorelines on matchup cards
- Live ticker strip at the top during gameweeks, cycling through all 5 matchups like a sports score bar
- Micro-animation (brief flash/count-up) when a manager’s live points tick up on poll refresh

10. Access / Login

- **No per-user authentication** — the site is public-by-link (or unlisted URL), viewable by anyone with the link, no accounts needed for the 10 managers
- **Single small “Admin” login button** in the top corner — only you (the admin) log in, giving access to any admin-only actions (e.g., re-triggering a data refresh, editing manager mappings if someone’s FPL ID needs fixing)
- This keeps the viewing experience frictionless (open link, see dashboard, no signup) while still giving you a way to manage the backend without touching the database directly

11. Extra Stats

- **Manager of the Week**: highest GW score across the league, called out on the dashboard each week
- **Streaks**: current win/loss streak per manager, shown on Standings and Manager Profile
- **Rivalry records**: head-to-head history between any two specific managers (all-time record within the season), visible from each Manager Profile — e.g., “3-1 all-time vs. [rival]”
- All computed from the same data already being pulled/stored (GW scores + our generated schedule), no extra API calls needed — just additional derived fields in the

standings/profile queries

12. Season Scope

- **Single season only** — no multi-season history/archiving needed for v1. Schema can stay flat (no season_id foreign keys everywhere); if a future season happens, that's a separate consideration down the line, not built now.

13. Status

All open questions resolved — PRD is locked and ready to build.